Tree Ensembles: Random Forests & Boosted Trees

ECON 8310: Business Forecasting — Lecture 5

Department of Economics

University of Nebraska at Omaha

August 25, 2026

Lecture Outline

From Single Trees to Ensembles

Averaging many imperfect models can outperform any single model.

The Wisdom of Crowds

A single expert can be wrong. Average many *independent* estimates and the errors tend to cancel.

> **Classic Example: Guessing the Ox's Weight**
>
> At a 1907 county fair, 800 people guessed the weight of an ox. No individual was very accurate — but the **average of all 800 guesses** came within one pound of the true weight.

The same principle transfers directly to models. A single decision tree is unstable and noisy — that was Lecture 4's closing complaint. Train many trees on different versions of the data and average them, and the noise largely cancels while the signal survives.

Ensembles work because independent errors cancel. And that word is doing real work: the *more independent* the individual predictions, the more variance averaging removes. Hold onto it — the entire difference between bagging and random forests turns on it.

Bagging: Bootstrap Aggregating

Train many trees on bootstrap samples; average their predictions.

Bagging: The Big Idea

Problem from Lecture 4: a single decision tree has very high variance — change a few training points and you get a completely different tree.

Bagging solution (Breiman 1996):

1. Draw B **bootstrap samples** (with replacement, same size as the original)
2. Fit one deep decision tree on each
3. **Average** all B predictions: $\hat{y}_{\text{bag}} = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{(b)}(\mathbf{x})$

Why it works. The variance of the averaged forecast is

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2,$$

where ρ is the pairwise correlation between tree predictions. The second term shrinks as B grows — so more trees never hurts. But the first term does not move at all.

That leaves a floor at $\rho\sigma^2$, set by how alike the trees are rather than by how many you grow. Averaging a thousand nearly identical trees buys you almost nothing, and that is the problem the next section solves.

Out-of-Bag (OOB) Error: Free Cross-Validation

Each bootstrap sample draws n observations *with replacement*, so it contains about **63%** of the distinct training points. The remaining 37% are **out-of-bag** for that tree — a free test set, different for every tree:

1. For each observation i, collect predictions only from the trees where i was out-of-bag
2. Average them to get $\hat{y}_i^{\text{OOB}}$
3. Compute the RMSE across all observations

Every one of those predictions came from a tree that never saw the observation — so the OOB RMSE is an honest out-of-sample estimate, and it lands close to 5-fold CV without refitting anything. In `sklearn` it is one argument: `RandomForestRegressor(oob_score=True)`.

> Below about 1,000 observations the OOB estimate gets noisy, because each observation is out-of-bag for too few trees. On a weekly business series — a few hundred rows — use proper walk-forward CV instead.

Random Forests: Decorrelating the Trees

Restrict each split to a random subset of features so trees are less alike.

The Problem Bagging Does Not Fully Solve

Recall the variance of the averaged forecast: $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. More trees shrink the second term to nothing. The first term never moves.

> **The correlation problem.** If one feature dominates — and in forecasting it always does, because last year's value is the best single predictor — then nearly every tree splits on it at the root. The trees end up near-identical, ρ stays large, and bagging buys you very little.

The Random Forest fix (Breiman 2001) is almost absurdly simple: at each split, consider only a random subset of $m < p$ features, and take the best split among *those*.

Sometimes the dominant feature is not in the candidate set, so the tree is forced to find its second-best story. Across hundreds of trees that produces genuinely different structures — ρ falls, and the floor falls with it. You deliberately make each individual tree *worse* in order to make the ensemble better.

Random Forest Hyperparameters

Parameter	sklearn name	Default	Effect
Number of trees	`n_estimators`	100	More = lower variance; plateaus ∼500
Max features/split	`max_features`	`'sqrt'`	Lower = more decorrelated
Max depth	`max_depth`	None	Limit only to reduce memory
Min samples/leaf	`min_samples_leaf`	1	Higher = smoother predictions
OOB score	`oob_score`	False	`True` gives a free CV estimate

Start with `n_estimators=500`, `max_features='sqrt'`, `min_samples_leaf=5`, `oob_score=True`. Note how few genuinely need tuning — `max_features` is the dial controlling ρ, and the rest is close to fire-and-forget. That is most of why random forests make a good default.

Set `random_state=42`: a forest is random twice over, in the bootstrap samples and in the feature subsets.

Feature Importance and Interpretation

Which predictors matter most? RF gives a data-driven answer.

Random Forest Feature Importance (Molnar 2022)

Mean Decrease in Impurity (MDI)

For each feature j, average the total reduction in node impurity across every split on j, weighted by the fraction of observations reaching that node. Normalize to sum to 1.

Feature (CA_1 FOODS, weekly)	Importance
units_lag4	0.45
units_lag1	0.18
units_lag52	0.11
snap_days	0.08
units_lag2	0.04
avg_price	0.04

Read it as a screening device, not an explanation. **MDI is biased** toward features with many split points. `permutation_importance` — shuffle a column, see how much accuracy falls — is the honest alternative.

Random Forest in Python

sklearn Code Template

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=500, max_features='sqrt',
min_samples_leaf=5, oob_score=True, random_state=42).fit(X_tr, y_tr)
print(rf.oob_score_);  importances = rf.feature_importances_
```

Model (CA_1 FOODS, 52-week test)	Test RMSE
Seasonal naïve	1,660
Single deep tree	1,415
Random Forest	**1,020**

The forest cuts RMSE roughly **28%** against a single deep tree, and is the first method in this course to beat the seasonal naive benchmark on this series.

What changed is not a better model of the series but a richer feature set — lags, price, and SNAP days are information ETS and ARIMA never had.

Gradient Boosting

Instead of averaging independent trees, fit each new tree to the errors of all previous trees.

Boosting vs. Bagging: The Core Difference

Bagging (Random Forest):

- Trees built **in parallel**, independently
- Each tree predicts y directly
- Averaging reduces *variance*
- More trees never reduces bias

Boosting:

- Trees built **sequentially**
- Each fits the *errors* of the ensemble so far
- Reduces *bias* as well as variance
- Will overfit if trees are too deep or too many

The intuition. Forecast 1 says \$50,000; actual is \$60,000, so the error is +\$10,000. Forecast 2 does not predict sales at all — it predicts *that error*, and says "the first model was short by \$10,000." Add them: \$60,000. Each stage corrects what remains, which is why boosting can fix a systematic bias that averaging never touches.

The Gradient Boosting Algorithm

Gradient Boosting (squared-error loss)

Initialize $F_0(\mathbf{x}) = \bar{y}$. For $b = 1, \ldots, B$:

1. Compute residuals: $r_i^{(b)} = y_i - F_{b-1}(\mathbf{x}_i)$
2. Fit a shallow tree T_b to $\{(\mathbf{x}_i, r_i^{(b)})\}$
3. Update: $F_b = F_{b-1} + \eta T_b$

Final prediction: $\hat{y} = F_B(\mathbf{x})$

The trees are deliberately **shallow** — depth 3 to 6 — because each one only has to capture a small piece of the remaining error. The ensemble converges slowly and on purpose.

The learning rate η is the main dial. Small values (0.01–0.1) take a tiny step per tree: slower, needs more trees, far more robust. Large values (> 0.3) move fast and risk overshooting. The reliable recipe is a small η, many trees, and **early stopping** — without which boosting will happily fit your noise.

XGBoost

Three improvements over vanilla boosting: second-order optimization, explicit regularization, and column subsampling.

XGBoost: Three Advances (Chen and Guestrin 2016)

1. **Newton step (second-order optimization).** Uses the gradient *and* the curvature of the loss to choose each tree's direction, so it converges in fewer trees than first-order boosting.
2. **Explicit regularization.** Penalizes the number of leaves (γ) and the magnitude of leaf weights (λ) directly in the objective — so overfitting is controlled by the loss function rather than only by early stopping.
3. **Column subsampling.** Borrowed straight from random forests: sample features per tree and per split, decorrelating the ensemble.

Notice that the second advance is *exactly* the Ridge and LASSO idea from next week, applied to leaf weights instead of regression coefficients. `reg_lambda` is an ℓ_2 penalty; `gamma` penalizes complexity the way ℓ_1 does.

XGBoost dominated Kaggle tabular competitions from 2014–2018. `LightGBM` is faster on large data; `CatBoost` handles categoricals natively.

XGBoost: Key Hyperparameters

Parameter	XGB name	Typical	Effect
Number of trees	`n_estimators`	500–2000	More + small η = better (use early stopping)
Learning rate	`learning_rate`	0.01–0.1	Smaller = more robust; needs more trees
Tree depth	`max_depth`	3–6	Shallow preferred; depth >6 risks overfit
Row subsample	`subsample`	0.7–0.9	Fraction of training rows per tree
Col. subsample	`colsample_bytree`	0.7–0.9	Feature fraction per tree (like RF)
L2 leaf penalty	`reg_lambda`	1	Ridge on leaf weights; stabilizes

Practical starting point: `learning_rate=0.05`, `max_depth=4`, `n_estimators=1000` with early stopping on validation RMSE. Tune `max_depth` and `subsample` via `TimeSeriesSplit`.

XGBoost in Python

Code Template

```
model = xgb.XGBRegressor(n_estimators=1000, learning_rate=0.05,
max_depth=4, subsample=0.8, colsample_bytree=0.8,
early_stopping_rounds=50, random_state=42)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)
```

`eval_set` is what makes `early_stopping_rounds` work: XGBoost watches validation RMSE and halts when it stops improving, which is how you get away with setting `n_estimators` far higher than you need.

The validation set must respect time. Build it with `TimeSeriesSplit` and hold out the final stretch as a true test set.

On macOS `xgboost` also needs OpenMP: `brew install libomp`, or the import fails.

Key Takeaways and Roadmap

Bagging reduces variance; boosting reduces bias. Together they cover most of what tabular forecasting needs.

Tree Methods: Which One When

Method	Training	Interpretable	Tuning	Accuracy
Decision Tree	Single tree	High	Low	Baseline
Random Forest	Parallel (B trees)	Medium	Low	Good
XGBoost	Sequential (B trees)	Low	Higher	Best

Random Forest is the better default: robust, almost no tuning, OOB error for free. **Move to XGBoost** when you can afford to tune and need the last few points — and budget for that tuning, because an untuned XGBoost often loses to a default forest.

Neither can **extrapolate** beyond the training range. A tree predicts the mean of a leaf, and no leaf contains values it never saw. On a trending series, both will flatten out exactly when you need them to keep climbing.

Lecture 5: Key Takeaways

1. **Ensembles** work because independent errors cancel. Averaging many imperfect models beats any single model.
2. **Bagging** averages B trees fit to bootstrap samples; variance falls as B grows but floors at $\rho\sigma^2$. **Random Forests** subsample features at each split to cut ρ — the key gain over plain bagging.
3. **OOB error** is free cross-validation: each observation sits out $\approx$37% of trees, giving an unbiased generalization estimate.
4. **Boosting** inverts the idea — trees are built sequentially, each fitting the ensemble's remaining errors. Bagging attacks variance; boosting attacks bias. **XGBoost** adds a Newton step, penalties on leaf count and weights, and column subsampling.
5. Tune by cross-validation, always with `TimeSeriesSplit` — a random split leaks the future into the past — and pair boosting with **early stopping**.

Next: Regularization & Model Selection — choosing among many predictors by shrinking coefficients rather than counting them.

References I

- Breiman, Leo (1996). "Bagging Predictors". In: *Machine Learning* 24.2, pp. 123–140.
- — (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32.
- Chen, Tianqi and Carlos Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- Molnar, Christoph (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd. Open access. URL: https://christophm.github.io/interpretable-ml-book/.